

实验 4 页式内存管理

截止时间：2026 年 6 月 17 日 23:59【需要提前拿成绩的应届毕业生，应至少要求提交成绩前一周提交实验——如果该时间早于 6 月 17 日，具体时间等候通知】

提交要求：

- 正常提交→OBE 上“实验 4”；
- 截止后覆盖提交→OBE 上新开条目（待 OBE 上创建）。

实验报告编制要求：

- 文件名：学号姓名.pdf ←注意
 - ◆ 必须使用 PDF 格式，不能是 doc、docx、odt、md 等格式
- 报告标题：体现出“实验 4”等信息
- 报告作者：学号 姓名 ←注意
- 不同题目之间有明显的间隔，要能够容易地区分不同题目对应的实验内容
- 在报告开篇说明新增的文件和修改过的原有代码文件（包括在前几次实验中编写但在此次实验中做出修改的代码）←注意
- 实验报告不做任何其他版式上的要求

提交要求：

- 提交完整可编译运行的源代码（提交前在 compile 目录执行 make clean 清楚二进制文件以减少体积）
 - ◆ 所有代码在提交前必须保证能在实验 2 所提及的服务器上编译通过
- 实验报告与源码目录一起压缩，压缩包必须使用 Linux 常用的压缩包格式.tar.gz，源码不用额外打包，源码目录与实验报告一起打包。
 - ◆ 假设源码目录包 Xinu-qemu-2026 和实验报告 123456 学生.pdf 在同一个目录
 - ◆ 在该目录中执行：tar czvf 学号.tar.gz .
 - ◆ 压缩包文件名为学号，上述命令最后有一个“.”

代码编写要求：

➤ **函数命名规则**

- ◆ 所有新增的**内核态**函数均以“**k 学号_**”作为前缀，例：学号为 123456 的同学实现了一个内核态 `printf` 函数，则该函数的函数名应为“`k123456_printf`”
- ◆ 所有新增的**用户态**函数均以“**u 学号_**”作为前缀，例：学号为 123456 的同学实现了一个用户态 `printf` 函数，则该函数的函数名应为“`u123456_printf`”
- ◆ **在本实验手册内提及的新函数均应按照上述要求命名**
- ◆ 此处所指的函数包括 C 语言函数和汇编代码函数

➤ **全局变量、结构体命名规则**

- ◆ 所有新增的全局变量和结构体的命名也遵循函数命名规则
 - ◆ 局部变量不用遵循上述规则
- 所有新增的其它结构体（如果需要）、全局变量声明（如果需要）、新的函数声明（如果需要），放到 `include/Lab4.h` 中，该头文件可在 `xinu.h` 中引用 **←注意**
- 所有新增的功能函数，放入 `system` 目录下以“`Lab4_`”为前缀名的源文件中；如果所有函数都在同一个源文件中，则该文件名称可为 `Lab4.c`；也可按照 Xinu 的习惯以函数本身的功能名称参与文件名，如 `Lab4_printf.c` **←注意**
- **对原有代码的修改**：单行改动，应以修改行前一行或当前行尾的注释“`// Lab4 学号`”标记；多行修改，应以首行行前“`/*Lab4 学号: Begin*/`”和末行行后“`/*Lab4 学号: End*/`”标记
- 在 `shell` 目录下新建一个文件 `xsh_lab4.c`，其中包含一个 `shell` 命令函数“`xsh_lab4`”，参数类型请参考其他 `shell` 命令，本实验的测试代码均实现在该源码文件中；`fork/exec` 相关的部分一般推荐在 `system` 目录下新建源码文件进行实现，文件名可参考其他文件命名方式
- 在 `shell/shell.c` 的 `cmdtab` 数组中，添加“`{“lab4”, FALSE, xsh_lab4}`”，注意保证语法正确（后续可在 Xinu 的 `xsh` 中通过 `lab4` 命令来测试所有题目）

本次实验**要求提交代码**（参看前文“提交要求”），也需提交实验报告。实验报告应说明对代码的修改（新增、修改等），以及**丰富的截图和配套的文字说明**（不能只有截图而无文字说明，或只有文字而无截图）。

1. 实验目的

希望学生能掌握页式内存和虚拟内存管理，将 Xinu 转化为一个更为现代化的操作系统内核。

2. 实验要求

【必做：2.1, 2.2, 2.3, 2.4】

2.1 实现支持区分内核态/用户态的进程隔离的页式内存管理机制

2.2 实现支持页式内存管理的用户态系统调用 `fork` 和 `exec`，两个函数的调用方式参考实验 2 中 `fork` 和 `exec` 的相关说明：`int fork(void)`和 `void exec(void*, pri16, char*, uint32, ...)`

2.3 实现用户态 shell 命令 `xsh_lab4`，应完成如下要求：

- a `xsh_lab4` 命令运行于用户态（应通过调试等方式展示该函数的指令运行于用户态）
- b `xsh_lab4` 中定义一个局部变量，然后输出该变量的地址，根据传入的参数调用自己实现的用于测试 `fork/exec` 的函数（名称自拟），最后输出学号和姓名（如果中文显示有误，可使用拼音）。如：

```
// 请自行保证代码正确性
int32 x;
printf("&x=0x%x\n", &x)
if (strcmp(argv[1], "1") == 0)
    fork_test1(潜在参数);
else
    fork_test2(潜在参数);
printf("<学号> <姓名>\n");
```

- c 不同场景下的 `fork_test` 应至少包含如下功能：

➤ 一种模式下，`fork` 得到的子进程不执行 `exec`，而是执行与父进程一

样的代码，该代码中至少输出当前进程 PID、进程名和某个局部变量或参数的地址；

- 另一种模式下，fork 得到的子进程执行 exec（需另提供一个子进程入口函数），父进程和子进程中至少输出当前进程 PID、进程名、所在函数名和某个局部变量或参数的地址）

- d 在 xsh_lab4 运行期间，查看内存映射情况(QEMU 控制台命令: info mem)
- e xsh_lab4 及各子进程正常执行结束后，正确销毁该进程占用的内存空间，将页目录、页表、栈等空间释放掉，以为其他进程所用
- f 在进程创建和销毁过程中，输出为其分配和释放的内存页物理地址（或页编号），并说明各个物理页的用途，结合 d 中的内存映射情况说明每个物理页在 lab4 进程中的逻辑地址
- g 再次执行相同的 lab4 命令，查看对比此次执行过程分配释放的内存页（及各页的用途）与上一次执行的区别与联系
- h 在非调试状态下，shell 中执行 lab4 命令及各子进程能够正常结束

2.4 实验 3 中的（用户态）“lab3”命令能够在页式虚拟内存环境中正常运行

- 可以不用改成 fork/exec 模式

2.5 （扩展要求，不强制）

- a 实现页式内存管理机制下的堆分配/释放函数，在 lab4 命令及各子进程中调用并观察结果；尝试在分配堆内存超过释放堆内存的行为下，在进程结束时全部释放
- b 实现用户栈扩展（参考课件第 10 章）

2.6 其他说明

- a 可以采用与课堂讲述的 Xinu-vm 参考实现不一样的实现方式，在实验报告中详细说明设计思路与实现细节（包括内存布局的设计——如果完全与 Xinu-vm 参考实现一致，内存布局的设计不用提供太多细节）
- b 内核态空间大小可以调整，但应保证不能将所有物理内存都与逻辑地址一对一映射，在实现过程中应该强制某些内存页采用动态映射方式并且保证测试过程中一定能触发动态映射（参考 Linux 的“高端内存”） ← 注意
- c 进程的用户栈空间可以位于高地址处，也可以位于低地址处（在实验报告中清晰说明包含用户栈空间的进程内存布局——图示+文字）

- d 为方便实现，**限制单个进程的用户栈空间不超过 4MB**
- e 原有 Xinu 中的内存初始化函数 `meminit` 和 `bufinit`，两者均在 `initialize.c` 中被调用，后者的调用直接注释掉，前者如果直接对该函数做修改，此处调用也应被注释掉，它的调用位置可参考课件 **←注意**
- f `nulluser()`中统计空闲内存的代码可以注释掉，或是修改使其适用 **←注意**
- g 由于虚拟内存的引入，在未完全实现之前，可以将系统的大部分功能先去掉，再根据所实现的功能逐步开放。如：一开始可以将 `nulluser()`中所有代码注释掉，添加少许代码验证虚拟内存是否正常开启；然后开启其他功能，验证新进程是否正确运行在虚拟内存空间（内核态进程、用户态进程）；检查进程正常运行结束，进程内存空间是否正确释放。【此处只是开展实验、编写代码的一种建议，不是强制要求】

3. 思维扩展

（不要求在实验报告中作答）

- 3.1 查阅资料，理解 64 位 CPU 架构下页表的组织形式，64 位虚拟地址如何映射到物理地址。
- 3.2 查阅资料，理解 Linux 的写时复制实现。
- 3.3 深入分析 HarmonyOS（LiteOS-a）和 xv6 的代码，查看它们怎么实现的页式内存管理。